Exam Software Testing & Verification 2017/2018

Th. 24th May 2018, 8.30–10.30, EDUC-ALFA

Lecturer: Wishnu Prasetya

1 Part I (5pt)

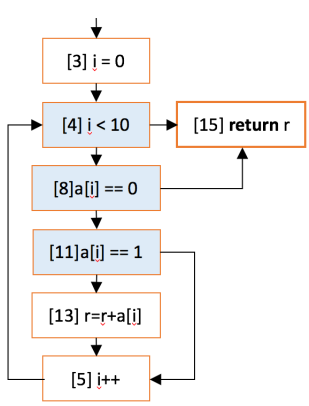
1. Prove that node coverage does **not** subsume edge coverage.

   Answer: Consider CFG consisting of the states $\{s_0, s_1, s_2\}$ with s_0 as the initial node, s_2 as the exit node, and as the edges: $s_0 \to s_1$, $s_0 \to s_2$, $s_1 \to s_2$. A single test path $[s_0, s_1, s_2]$ would cover all nodes, but miss the edge $s_0 \to s_2$.

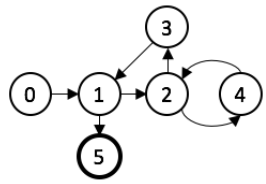
2. Consider the program below. Draw its corresponding CFG. Annotate each node in your CFG with the expression/statement that the node represents.

```
1  addme(r,a) {
2    for(
3        int i=0;
4        i<10    ;
5        i++     ) {
6
7      if
8         (a[i]==0)
9          break ;
10     if
11        (a[i]==1)
12        continue ;
13     r = r + a[i]
14   }
15   return r
16 }
```

   Answer:



3. Consider the following CFG G of some program. Give all prime paths in G *that pass through* the node 3.

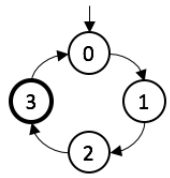


Answer: There are six, namely:

three elementary cycles that contain 3: [1,2,3,1], [2,3,1,2], and [3,1,2,3],

and three non cycle simple paths: [0,1,2,3], [4,2,3,1,5], and [3,1,2,4].

4. Consider the following CFG G of some program. Node 0 is the starting node, and 3 is the exit node.



How many prime paths does G have? Give a minimum sized test set T that would give full prime path coverage.

Answer: There are four prime paths. Each one is a cyclic simple path, starting in different nodes in G.

A single test path $[0, 1, 2, 3, 0, 1, 2, 3]$ will cover all these prime paths.

5. Consider a program $P1(x, y, z)$. The domains of x, y, and z are partitioned as follows:

x	is partitioned into three blocks:	X_1, X_2, X_3
y	is represented by just one block:	Y
z	is partitioned into two blocks:	Z_1, Z_2

Give a smallest **test set** that would give full Pair-wise Coverage (PWC) over the above blocks.

Represent test cases by triples of blocks. For example, a test case $tc_1 = (X_1, Y, Z_1)$ represents a test case where a value from the block X_1 is chosen for x, a value from Y for y, and a value from Z_1 for z.

Answer:

(X_1, Y, Z_1)
(X_1, Y, Z_2)
(X_2, Y, Z_1)
(X_2, Y, Z_2)
(X_3, Y, Z_1)
(X_3, Y, Z_2)

6. Consider a program $P2(x, y, z)$. The domains of x, y, and z are partitioned as follows:

x	is partitioned into these blocks:	X_1, X_2, X_3
y	is partitioned into these blocks:	Y_1, Y_2, Y_3
z	is partitioned into these blocks:	Z_1, Z_2, Z_3

Two test cases have been selected as the base tests, namely $tc_1 = (X_1, Y_1, Z_1)$ and $tc_2 = (X_2, Y_2, Z_2)$. Give the smallest **test set** that would give full Multiple Base Choice Coverage (MBCC).

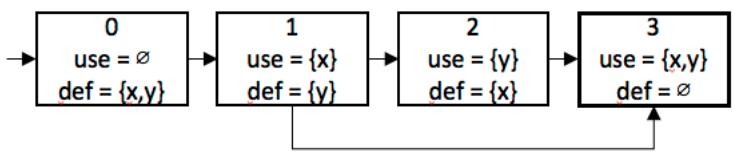
Answer: The test set with the following test cases:

(X_1, Y_1, Z_1) (base test)
(X_3, Y_1, Z_1)
(X_1, Y_3, Z_1)
(X_1, Y_1, Z_3)
(X_2, Y_2, Z_2) (base test)
(X_3, Y_2, Z_2)
(X_2, Y_3, Z_2)
(X_2, Y_2, Z_3)

7. One of the coverage concepts offered by data-flow oriented testing is *All Uses Coverage* (AUC). Give the definition of AUC. If you define this in terms of def-path sets or def-pair sets, define these concepts as well.

   Answer: For every variable x in the program under test, and for each def-pair $du(i, j, x)$, we add to the TR of AUC a member of this def-pair. The set $du(i, j, x)$ consist of all du-paths of x that starts at the node i and ends at the node j.

8. Consider the following CFG of some program. The nodes are annotated with defs and uses information.



   Give a test requirements (**TR**) that corresponds to full *All Defs Coverage* (ADC). Specify which requirement belong to which variable.

   Answer:

   The TR contains:

   For x: one of $\{01, 013\}$ and 23.

   For y: one of $\{12, 123, 13\}$.

9. Consider again the program in No. 8. Give a test requirements (**TR**) that corresponds to full *All Uses Coverage* (AUC). Specify which requirement belong to which variable.

   Answer:

   The TR contains:

   For x: 01, 013, and 23.

   For y: 12 and one of $\{123, 13\}$.

10. Consider a program $Q(s)$ that takes a string s as a parameter. The string is a simplified HTML whose format is defined by the grammar below (in the BNF notation):

S	$\rightarrow$	ϵ	(RuleS1)
S	$\rightarrow$	$Part$	(RuleS2)
S	$\rightarrow$	$Text$	(RuleS3)
$Part$	$\rightarrow$	"<*div*>" S "</*div*>"	(RuleP)
$Text$	$\rightarrow$	"*a*" S	(RuleT)

   The non-terminals are $S, Paragraph$, and $Text$ with S as the starting symbol. Symbols between quotes are terminals.

A tester wants to test the program Q using the string "$<div>a<div>aa</div></div>$" as the input. Prove that the string is a valid string in the above grammar. Prove this by showing a **derivation** that leads to this string.

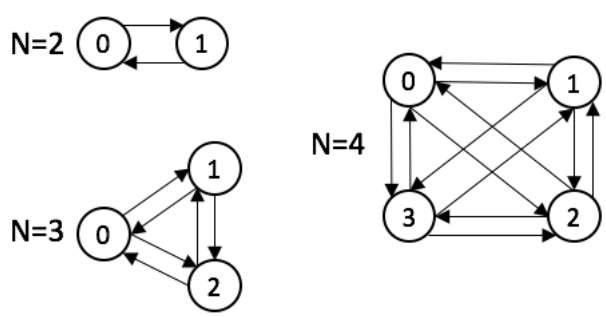
Answer: The derivation is shown below.

S
$\rightarrow$ (using RuleS2)$Part$
$\rightarrow$ (using RuleP)$<div>\ \ S\ \ </div>$
$\rightarrow$ (using RuleS3)$<div>\ \ Text\ \ </div>$
$\rightarrow$ (using RuleT)$<div>\ \ a\ S\ \ </div>$
$\rightarrow$ (using RuleS2)$<div>\ \ a\ Part\ \ </div>$
$\rightarrow$ (using RuleP)$<div>\ a\ <div>\ \ S\ \ </div>\ </div>$
$\rightarrow$ (using Rule3)$<div>\ a\ <div>\ \ Text\ \ </div>\ </div>$
$\rightarrow$ (using RuleT)$<div>\ a\ <div>\ \ a\ S\ \ </div>\ </div>$
$\rightarrow$ (using RuleS3)$<div>\ a\ <div>\ \ a\ Text\ \ </div>\ </div>$
$\rightarrow$ (using RuleT)$<div>\ a\ <div>\ \ a\ a\ S\ \ </div>\ </div>$
$\rightarrow$ (using RuleS1)$<div>\ a\ <div>\ a\ a\ \epsilon\ </div>\ </div>$

2 Part II (4pt)

Answer: 0.6 pt for every sub-question, except 1a: 4pt.

1. A full CFG of size N represents a program with N nodes whose control flow is most complex among programs with the same number of nodes. A CFG is *full* if every pair of distinct nodes n and n' in the CFG are connected by edges $n \rightarrow n'$ and $n' \rightarrow n$. Self-edges of the form $n \rightarrow n$ are excluded. The picture below shows three full CFGs with N =2, 3, and 4. We assume that each CFG has a single starting node and a single exit node.



For example, the CFG for $N = 2$ has two prime paths.

Questions:

(a) How many prime paths does each of the other CFGs above have?

Optional bonus: (hard, 0.5 pt) if you can give a general formula for any $N \geq 2$.

Answer: First, note that there are no non-cyclic prime path in such a CFG (in other words, all prime paths are cyclic). So, it comes down to counting how many different cycles the CFGs have.

i. $N = 3$
The CFG would have cycles of length 4 and cycles of length 3. Cycles of length 4 will have to go through all elements in $\{0, 1, 2\}$. Note that a cycle will have the form of $s{+}{+}[v]$ where s is the prefix and v is the same as the first element of the

prefix s. Now notice that s must be a sequence of all different elements. In this case have to go through all elements in $\{0, 1, 2\}$, in some order. In other words, s is a permutation of $\{0, 1, 2\}$. Since the graph is full, we can traverse any permutation. So, the number of cycles of length 4 that go through all elements in $\{0, 1, 2\}$ is the same as the number of permutations of this set, and that is 3!.
Similarly, there are 2! cycles of length 3 that go through all elements of $\{0, 1\}$, and another 2! cycles for $\{1, 2\}$, and another 2! for $\{1, 2\}$.
There is no cycles of length two because we exclude self-cycles (a direct cycle from v to v).
So in total, for $N = 3$, there are $3! + 3 * 2! = 12$ cycles, hence also the same number of prime paths.

ii. For $N = 4$, there is one subset of size 4, 4 subsets of size 3, and 6 subsets of size 2. The number of unique cycles is:

$$1{*}4! + 4{*}3! + 6{*}2! = 24 + 24 + 12 = 60$$

In general, in a full CFG of size N ($N{\geq}2$), there are $\binom{N}{k}$ subsets of size k in the CFG, $1{<}k{\leq}N$. Each such subset V_k would generate $k!$ permutations, and hence also $k!$ cycles of size $k + 1$ that go through all the elements in V_k.

So, in total there are this number of prime paths:

$$\sum_{k=2}^{N} \binom{N}{k} * k! \;=\; \sum_{k=2}^{N} \frac{N!}{k! * (N-k)!} * k! \;=\; \sum_{k=2}^{N} \frac{N!}{(N-k)!}$$

(b) What is theoretically the minimum size of the test set that is needed to cover all those prime paths for each of the CFGs above?

Answer: Since all the nodes are connected, you can always construct a path that tour any given finite set of paths. So the answer is: "one".

Note: such a test case would represent a pretty long execution and might not be the best choice to be used as a test case. For one thing, it would be more difficult to design. Secondly, due to its length, it would also be harder to debug, in case it fails. Shorter test cases would be preferred, although that may mean you have to write more of them.

2. Consider a program with 3 parameters: $P(a, b, c)$. We will divide the domain of a into k blocks, named: $A_1 ... A_k$, where $k{\geq}2$. The domain of b is divided into two blocks, named B_1 and B_2. The domain of c is also divided into two blocks, named C_1 and C_2.

(a) Suppose we want to do pair-wise testing over those blocks. How many pairs of blocks have to be covered in total?

Answer:

We have:

$2k$ pairs over the domain of $a \times b$.

$2k$ pairs over the domain of $a \times c$.

4 pairs over the domain of $b \times c$.

So, in total there are $4k + 4$ pairs to cover.

(b) Give a minimal test set that would deliver full Pair-wise Coverage (PWC) over the blocks.

Answer: Consider first the following test set which would cover all pairs among the blocks A_1 and A_2 and the blocks of b and c.

$$T = \begin{array}{c} (A_1, B_1, C_1) \\ (A_1, B_2, C_2) \\ (A_2, B_1, C_2) \\ (A_2, B_2, C_1) \end{array}$$

We still need to cover all pairs that concern the other blocks of a (so, $A_3, A_4, ...$). Since T already covers all pairs between b and c, we only need to worry about covering the pairing between these remaining A-blocks with the other two parameters (we don't have to worry about covering the pairing between b and c anymore).

For A_3 we now add these two test cases:

$$U_3 = \begin{array}{c} (A_3, B_1, C_1) \\ (A_3, B_2, C_2) \end{array}$$

With add similar test cases for the remaining $A_4, A_5, ...$

The test set $T \cup U_3 \cup U_4 ... \cup U_k$ will cover all the pairs, and is minimal. It has in total $4 + 2 * (k - 2) = 2k$ test cases.

3. Consider the program fragment below. Assume that all array accesses are valid.

```
1  module A {
2    int calculateTotal(int N) {
3       int[] a = getTransactions() ;
4       int total = 0 ;
5       int k = 0 ;
6       while (k<N) {
7          increase(a,k) ;   // call point (*)
8          if (k<=1900)
9             limit = 10000
10         else
11            limit = 20000
12         if (total < limit)
13             total = total + a[k]
14         k++ ;
15      }
16      return total ;
17   }
18 }
19
20 module B {
21  increase(int[] a, int k) {
22    if (k<=1900)
23       a[k] = a[k] + 1 ;
24    else
25       a[k] = a[k] + 2 ;
26    a[k] = 100*a[k] ;
27    if(a[k]<0)
28       a[k] = 0 ;
29  }
30 }
```

The def and use of an array a is defined as follows:

- An assignment a =... is a def of a.
- An assignment a[e] = ... is a def of a.
- An expression a[e] in the guard of an **if** or a **while** is a use of a.
- An expression a[e] in the right-hand-side of an assignment is a use of a.

Furthermore:

- The array a is passed 'by reference' to the program **increase**. This implies that both programs above read from and write to the same array a.

Your task:

(a) What are the coupling variables between the method calculateTotal and increase.

(b) List all coupling paths between the method calculateTotal and increase. **For each path, mention the name of the coupling variable that the path is associated to.** Use line numbers to describe your paths. For example: [2,3,4,5,6] describes the path from the start of calculateT

otal$uptoitsloop - head$.

(c) Which paths should be included as your test requirements (**TR**) for achieving full **All Coupling Uses Coverage** (ACUC)?

Answer: Coupling variables: a and k. All the coupling paths are listed below:

coupling var a:	ρ_1 = 3..6[true]..**7**..22[true] 23
	ρ_2 = 3..6[true]..**7**..22[false] 25
	ρ_3 = 26 27[false] 29 **7** 8[true] 9 12[true] 13
	ρ_4 = 26 27[false] 29 **7** 8[false] 11 12[true] 13
	ρ_5 = 28 29 **7** 8[true] 9 12[true] 13
	ρ_6 = 28 29 **7** 8[false] 11 12[true] 13
coupling var k	τ_1 = 5..6[true]..**7** 22
	τ_2 = 14..6[true]..**7** 22

The following TR will do for ACUC:

for a: $\{\rho_1, \rho_2\}$ plus one of $\{\rho_3, \rho_4\}$ plus one of $\{\rho_5, \rho_6\}$

for k: both τ_1 and τ_2.

3 Part III (1pt)

Answer: 1a 0.3pt, 1b 0.4pt, 2 0.3pt

1. Consider the following binding triples within `foo`(), and their corresponding coupling paths. In all cases, the context variable is o. All named paths are distinct.

binding triples			
coupling sequence	runtime type of context-var	coupling variables	coupling paths
C_1	A	$\{o.x\}$	σ_1
C_1	B	$\{o.x\}$	σ_2
C_2	A	$\{o.x, o.y\}$	τ_1 and τ_2 for $o.x$ τ_3 for $o.y$
C_2	B	$\{o.z\}$	τ_4

   (a) Give the test requirement (TR) for achieving full full All-Poly-Classes (APC) coverage.

   Answer: $\{\sigma_1, \sigma_2, \tau_4\}$ plus one of $\{\tau_1, \tau_2, \tau_3\}$.

   (b) Let TR_P be the test requirements (**TR**) to get full All-Poly-Classes (APC) coverage, and TR_{DU} be the **TR** to get full All-Coupling-Defs-Uses (ACDU). Note that there might be multiple possible TR_Ps and TR_{DU}s.

   Question: which target paths **must be in both** TR_P and TR_{DU}? (in other words: give the coupling paths that are guaranteed to be in $TR_P \cap TR_{DU}$).

   Answer: For APC we need: $\{\sigma_1, \sigma_2\}$ plus one of $\{\tau_1, \tau_2, \tau_3\}$ plus τ_4.

   For ACDU we need: one of $\{\sigma_1, \sigma_2\}$ plus one of $\{\tau_1, \tau_2\}$ plus τ_3 plus τ_4.

   So, only τ_4 is guaranteed to be in the intersection.

2. Consider a program $Q(s)$ that takes a string s as a parameter. The string has to follow the grammar shown below (described in the BNF notation):

$$
\begin{array}{lcll}
S & \rightarrow & \epsilon & \text{(RuleS1)} \\
S & \rightarrow & \text{"}x\text{"} & \text{(RuleS2)} \\
S & \rightarrow & B & \text{(RuleS3)} \\
B & \rightarrow & \text{"<"}\ S\ \text{">"}\ S & \text{(RuleB)}
\end{array}
$$

   Give a test set for Q that would give full *Each Rule-Rule Coverage* (ERRC) with respect to the above grammar.

   Each test case has the form of a string, and <u>should be shorter than 11 characters</u> long.

   We favor a larger test set with shorter test cases.

   Answer: In any case we have the combination $RuleS3$; 1 $RuleB$ to cover. Any test case whose derivation ever use $RuleS3$ will cover this combination (in other words, if the test case contains " < ... > ...", it will cover the combination).

   For $RuleB$, these combinations have to be covered:

$$
\begin{array}{l}
RuleB;\ 1\ R \\
RuleB;\ 3\ R
\end{array}
$$

   where R is one of $\{RuleS1, RuleS2, RuleS3\}$. So in total there are 6 combinations to cover.

   The following test cases will cover them all:

" <> "	covers	$RuleB$; 1	$RuleS1$	and	$RuleB$; 3 $RuleS1$
" $< x > x$"	covers	$RuleB$; 1	$RuleS2$	and	$RuleB$; 3 $RuleS2$
" <<>><> "	covers	$RuleB$; 1	$RuleS3$	and	$RuleB$; 3 $RuleS3$